Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

Recapitulare notiuni C/C++. Mediul Visual C/C++ 6.0. Complexitatea algoritmilor.

1. **Recapitularea unor notiuni C. Limbajul C++.**
2. **Introducere mediul Visual C/C++ 6.0.**
3. **Complexitatea algoritmilor – algoritmi de sortare.**

1. Recapitularea unor notiuni C. Limbajul C++.

Informatii introductive

La laboratorul de "Structuri de date si algoritmi" veti lucra sub mediul Visual C++ 6.0. Programele vor fi scrise in C, dar vom folosi citeva facilitati specifice limbajului C++ care vor fi expuse in continuare. Fisierele sursa vor avea extensia .CPP (de exemplu STUDENT.CPP).

Iata deja o prima simplificare: doua semne // arata inceputul unui comentariu. Acest tip de comentariu incepe cu // si se termina la sfirsitul liniei.

Expresii si valori stinga

In C ne putem referi la un obiect folosind o expresie "valoare-stinga". O expresie valoare-stinga" este o expresie care poate sa apara in STINGA unei atribuiri. De exemplu numele unei variabile este o expresie valoare-stinga in timp ce rezultatul unei operatii aritmetice nu este valoare-stinga.

```
int i;
int v[10];
```

Valori stinga: `i` `v[i]` `v[i+1]`
Nu sint valori stinga: `i+1` `2*v[i]`

In C prin combinarea numelor de variabile cu anumiti operatori se obtin valori-stinga:

Operatorul *

Se aplica numai pointerilor, rezultatul fiind o valoare-stinga care se refera la obiectul a carui adresa este continuta de catre pointer:

```
int* p;        // valori stinga:
p = &i;       // p       nume_variabila
*p=5;        // *p      * pointer
```

Observati ca expresia *p (rezulatata din combinatia numelui lui p cu operatorul *) apare in stinga atribuirii.

Operatorul []

Se aplica numelor de tablouri si pointerilor, rezultatul fiind o valoare-stinga care se refera la obiectul al *n*-lea din tablou:

```
int tab[10];
int* p = &tab[0];       // valori stinga:
tab[2] = 3;             // nume_tablou [ index ]
p[2] = 4;               // pointer [ index ]
```

Mai sus, pointerul ***p*** este initializat cu adresa primului element din vectorul ***tab***. Expresia ***p[2]*** va referi al doilea element din vectorul a carui adresa este memorata in pointerul ***p***, deci ***tab[2]***.

Operatorii . si ->

Apar in legatura cu structurile si vor fi tratati putin mai tirziu.

Structuri

1. Definire

O structura este un tip de date nou. Atunci cind definim o structura, trebuie sa specificam numele structurii si cimpurile ei:

```
struct student {
   char* nume;
   int   nota;
   };
```

Am introdus tipul "*struct student*". Pentru a evita repetarea lui "*struct*" putem sa introducem un pseudonim pentru tipul "*struct student*" si anume "*Student*" astfel:

```
typedef struct student  Student;
```

Cele doua declaratii de mai sus pot fi comprimate in una singura:

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

```
typedef struct {
   char* nume;
   int   nota;
   } Student ;
```

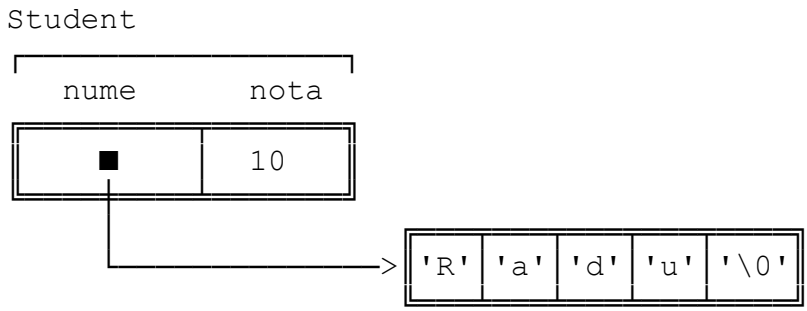
In C++, tipul definit de declaratia:

```
struct Student {
   char* nume;
   int   nota;
   };
```

poate fi denumit "struct Student" sau, doar simplu, "Student". In continuare ne vom folosi de aceasta facilitate care mareste lizibilitatea programelor. Repetam: programele vor avea extensia .CPP .

2. Obiecte de tip structura

Am definit structura *Student* avind CIMPURILE "*nume*" (adresa unui sir de caractere) si "*nota*" de tip int.



Odata definit tipul, acesta poate fi folosit pentru a declara variabile:

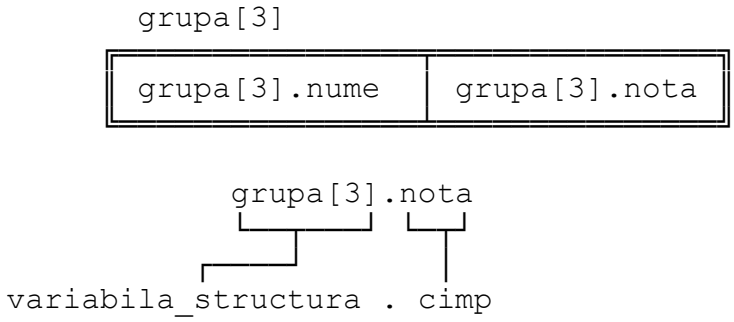
```
Student s1, s2;       // s1 si s2 sint doua variabile de tip Student
Student grupa[5];     // un tablou de variable Student
Student* ps;          // o variabila pointer la Student
Student* idx[5];      // o variabila tablou de pointeri la Student
```

3. Operatorii . si ->

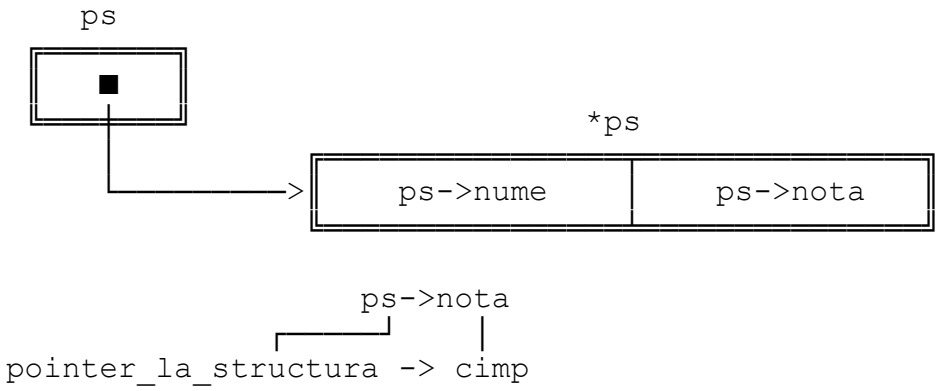
Folosirea cimpurilor unei structuri se face NUMAI CU REFERIRE LA UN OBIECT de tipul respectiv. Obiectul este referit printr-o valoare stinga care semnifica obiectul structura sau adresa obiectului structura.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

Operatorul **.** cere in stinga sa o valoare stanga de tip structura iar in dreapta, numele cimpului selectat, rezultatul fiind o valoare-stinga care se refera la cimpul selectat. De exemplu, din declaratiile de mai sus, cimpurile variabilei *grupa[3]* vor fi denumite: *grupa[3].nume* si *grupa[3].nota*



Operatorul **->** cere in stinga sa o expresie de tip "pointer la structura" iar in dreapta numele cimpului selectat, rezultatul fiind o valoare-stinga care se refera la cimpul selectat:



Exercitii

Avind urmatoarele declaratii:

```
int i, *pi;
Student s;
Student* ps;
Student ts[5];
Student* tps[5];
```

numiti tipurile urmatoarelor expresii. Decideti daca sint valori stinga sau nu:

```
i+2          pi          i            p+3
ts[2]        ts          ps           ps[2]
ps->nume    *pi          *(pi+2)     *pi+2
```

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

```
(ps+3)->nume   ps->nume[2]     tps                tps[2]
 tps[2]->nume  tps[2]->nume[2]
```

Pointeri

1. Definire

Pentru un tip de date T o variabila "*pointer la T*" se defineste astfel:

```
T* ptrT;          // ptrT este un pointer la T
```

O variabila pointer la T poate retine adresa unui obiect de tip T.

Exemple:

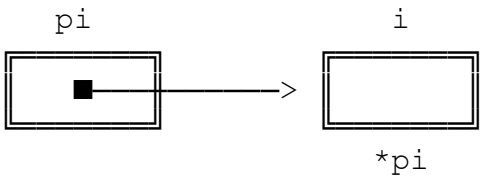
```
int* pi;          // pi este un pointer la int
char* tab;        // tab este un pointer la char
Nod *nou, *cap;   // nou si cap sint pointeri la tipul Nod
```

2. Initializare

Prima operatie care se face cu un pointer este initializarea. Un "*pointer la T*" poate fi initializat cu:

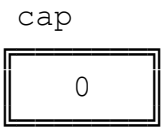
a) adresa unui T (care exista)

```
pi = &i;
```



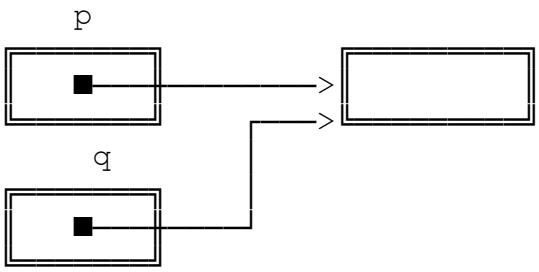
b) valoarea 0 (sau NULL) care semnifica adresa invalida

```
cap = 0;
```



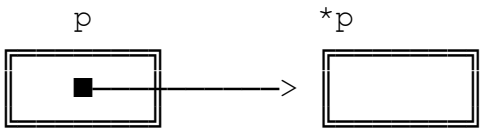
c) valoarea altui "*pointer la T*". De exemplu: daca *p* si *q* sint de tip *T** si *p* contine adresa unei variabile de tip *T* (a fost initializat in prealabil), atribuirea *q* = *p* va face ca ambii pointeri sa indice aceeasi variabila.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1



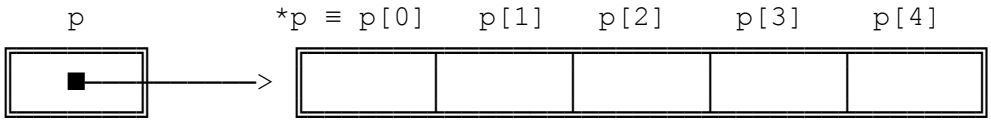
d) adresa unui spatiu de memorie alocat in zona de alocare dinamica. Spatiul alocat poate sa contina un singur obiect de tip T, acesta se exprima:

in C: `p = (T*) malloc( sizeof(T) );`
in C++: `p = new T;`



sau poate sa contina mai multe (n) obiecte de tip T:

in C: `p = (T*) malloc( n*sizeof(T) );`
in C++: `p = new T[n];`



Exprimarile din C++ sint in mod evident mult mai simple, si le vom folosi pe acestea in continuare.

Iata un exemplu:

```
typedef Student* PStudent;
PStudent* ptps;// pointer la tablou de pointeri la studenti
ptps = new PStudent[nr];
```

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

3. Dereferentierea

Este operatia prin care avind un "*pointer la T*" (care contine adresa unui T) obtinem o valoare stinga care se refera la obiectul pointat (vezi operatorul *).

Pentru a obtine obiectul pointat folosim operatorul * astfel:

```
*pi = 5;          // obiectul pointat ia valoarea 5
```

!!! Atentie !!!
Aceasta operatie poate fi aplicata numai pointerilor care contin intr-adevar adresa unui obiect. De exemplu nu puteti face dereferentierea unui pointer nul (cu valoare 0) sau a unui pointer neinitializat. Este valabil si pentru operatorul -> care contine si el o dereferentiere care se observa in scrierea echivalenta: ps->nota este echivalent cu (*ps).nota

4. Pointeri si tablouri

Numele unui "*tablou de T*" este convertit automat la tipul "*pointer la T*", deci poate fi folosit pentru a initializa un "*pointer la T*". Valoarea acestui pointer este adresa primului element al tabloului:

```
T tab[10];
T* ptrT = tab;    // ptrT contine adresa primului element
```

Un "*pointer la T*" este deseori folosit pentru a se referi pe rind la elementele unui tablou. Urmatoarele operatii semnifica:

```
ptrT++    // pointeaza la urmatorul element din tablou
          // creeaza o valoare stinga!
ptrT--    // pointeaza la elementul precedent din tablou
          // creeaza o valoare stinga!
```

5. Eliberarea spatiului alocat dinamic

Daca un pointer a fost initializat cu adresa unui spatiu din zona de alocare dinamica, atunci cind nu mai avem nevoie de spatiul respectiv (adica nu mai avem nevoie de obiectul din spatiul respectiv) vom elibera spatiul. El va putea fi astfel utilizat pentru alocari ulterioare.

Daca *p* este un pointer care a fost initalizat printr-o alocare de memorie, eliberarea memoriei alocate se exprima:

in C: `free(p);`
in C++: `delete p;`

!!!Atentie!!!
Nici *free()* nici *delete* nu modifica valoarea pointerului *p* dar obiectul a carui adresa este continuta de *p* nu trebuie sa fie referit dupa eliberare.

6. Observatie

In laboratoarele urmatoare, vor exista cazuri in care anumiti pointeri nu sint varibile simple, ci vor fi componente ale unor structuri de date complexe. Toate regulile de mai sus se pastreaza.

Referinte

Cind vrem ca o functie, atunci cind este apelata, sa modifice valoarea unei variabile din functia apelanta, trebuie sa trimitem ca argument un pointer la acea varibila. De exemplu, o functie care interschimba valoarea a doi "*pointeri la student*" va trebui sa primeasca ca parametri doi "*pointeri la pointeri la student*":

```
void Schimba(Student** unu, Student** doi)
{
     Student* trei;

     trei = *unu;
     *unu = *doi;
     *doi = trei;
}
```

Daca argumentele au tipuri mai complicate, atunci sintaxa din interiorul functiei devine greoaie. Pentru a rezolva aceasta problema putem trimite ca argumente REFERINTE. Un argument referinta trebuie interpretat ca fiind un PSEUDONIM pentru argumentul pasat. Orice modificare a referintei se va face asupra argumentului pasat:

```
void Schimba(Student*& unu, Student*& doi)
{
     Student* trei;
     trei = unu;
     unu = doi;
     doi = trei;
}
```

Observati ca sintaxa din interiorul functiei a devenit mai simpla.

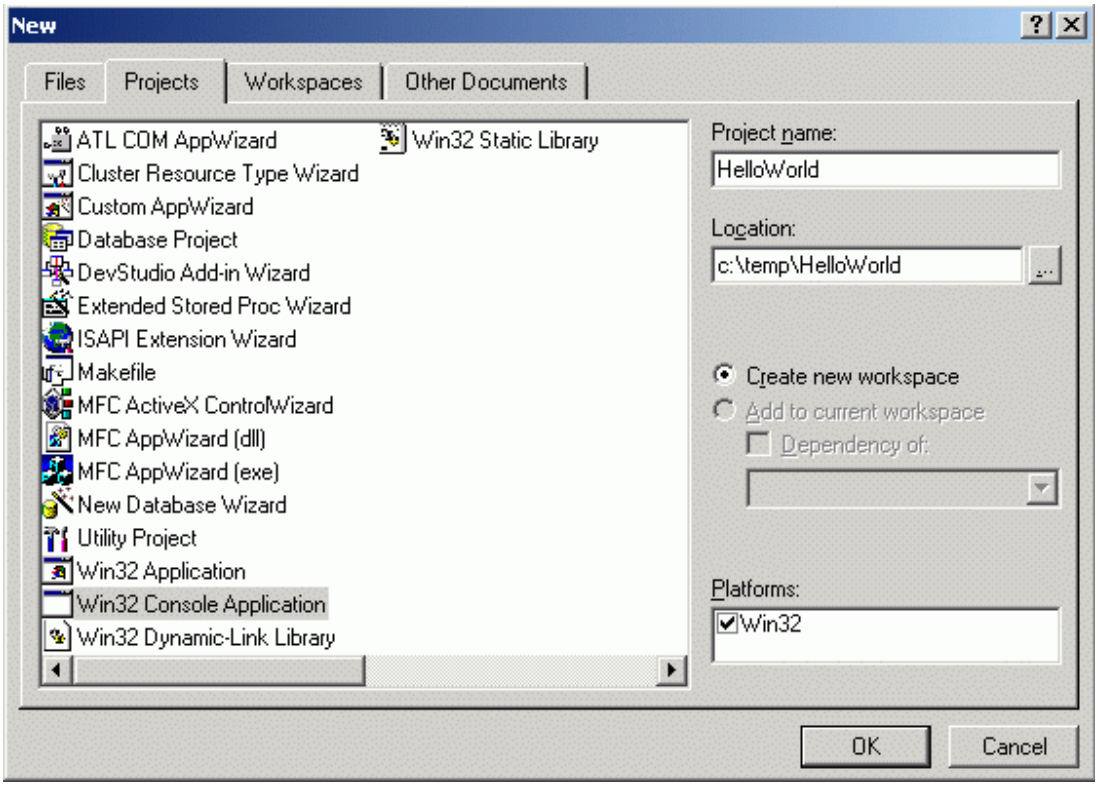
2. Prezentarea mediului Visual C/C++ 6.0

Visual C versiunea 6 este un puternic mediu integrat (IDE) pentru dezvoltarea aplicatiilor in C/C++ pentru platforma Windows. In cadrul acestui laborator vom dezvolta aplicatii consola, si nu aplicatii Windows native.

2.1. Crearea unui proiect

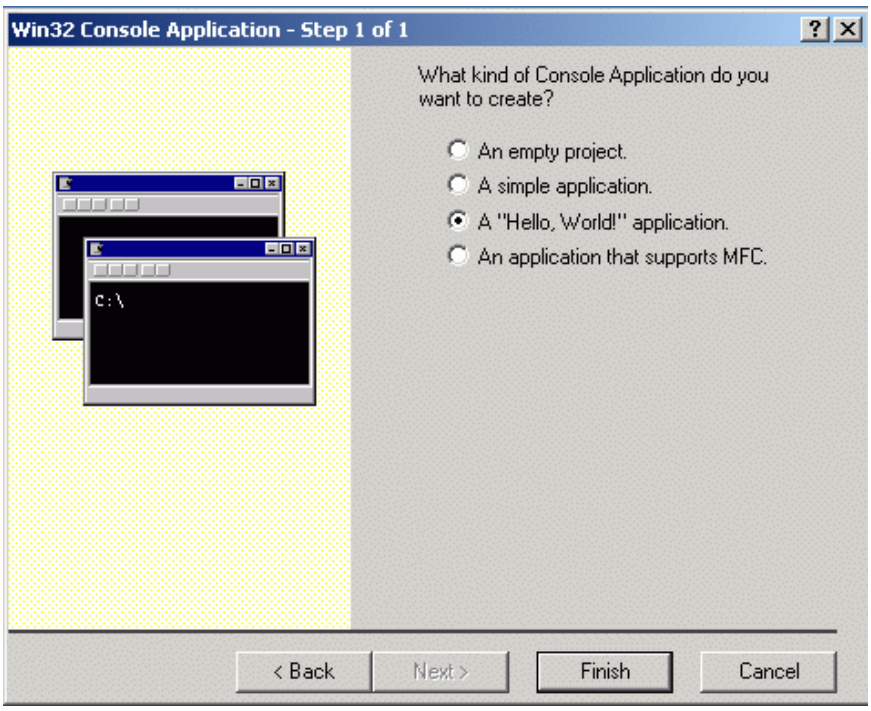
Pentru crearea unei aplicatii in Visual C este necesara existenta unui proiect. Nu se pot compila fisiere de sine statatoare. Vom incepe cu prima aplicatie in Visual C, clasica "Hello World".

Din meniul "File/New" se pot creea noi entitati: diferite tipuri de fisiere, tipuri de proiecte sau spatii de lucru (un "workspace" este o colectie de proiecte). Noi vom alege un nou proiect, o aplicatie consola (Win32 Console Application). Campurile "project name" si "project location" trebuiesc completate. In mod implicit, proiectul nou va fi asocial unui workspace nou cu acelasi nume ca si proiectul:

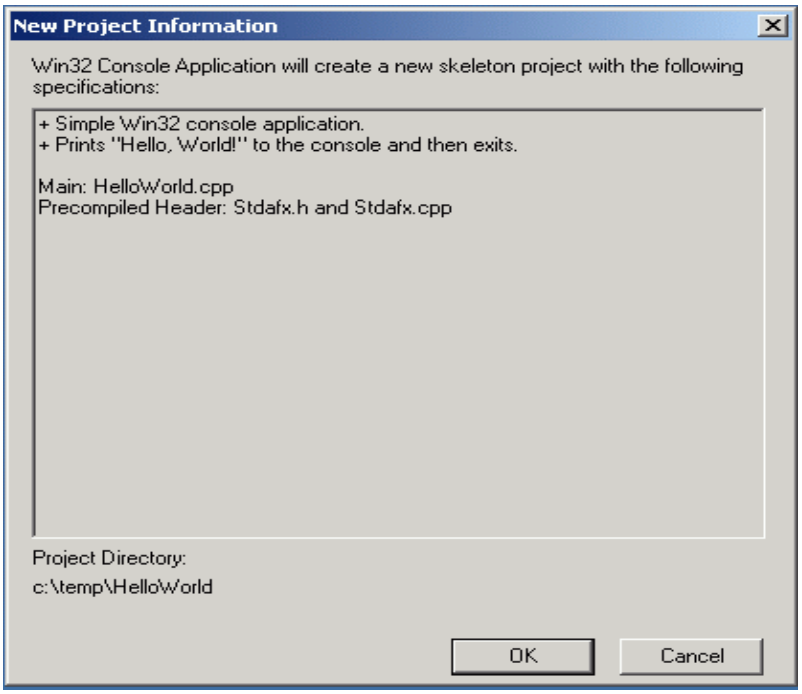


Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

Al doilea pas al “wizard-ului” consta in alegerea dintre cateva tipuri predefinite de aplicatii. In functie de alegere se generaza o parte din codul necesar aplicatiei, precum si stabilirea unor directive de compilare.

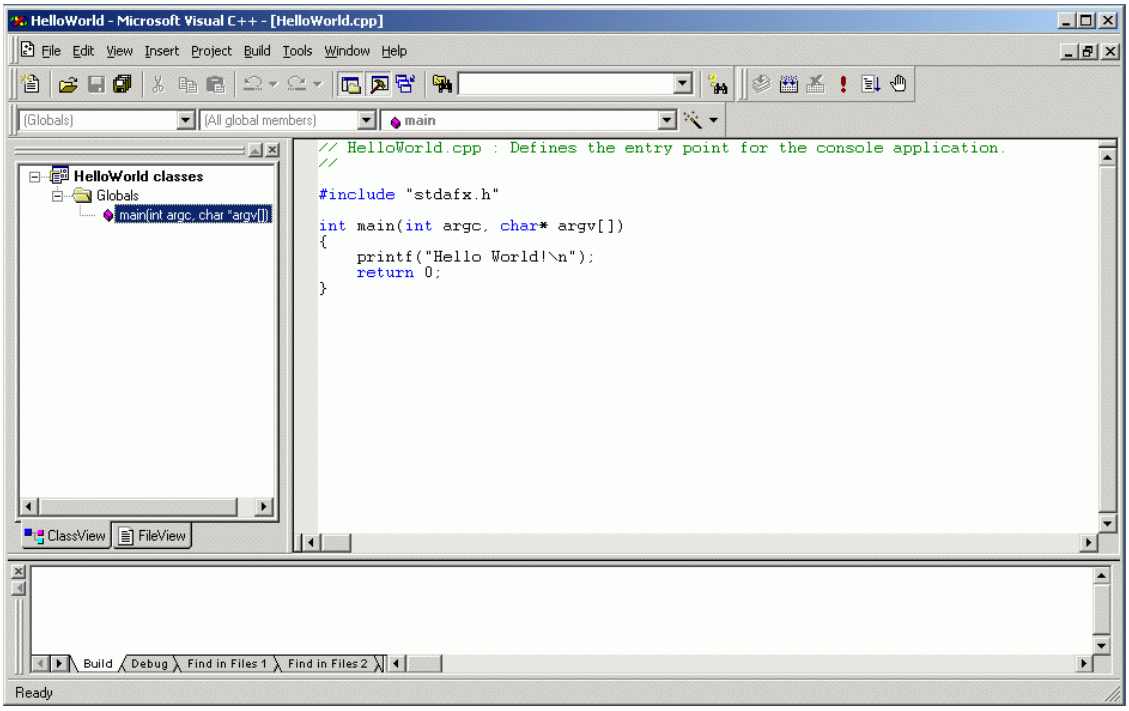


Ultimul pas este doar de verificare; ne sunt oferite detalii despre ceea ce va urma sa contina proiectul:



"Precompiled Header" ar putea fi tradus prin "fisier header precompilat". Este o tehnologie proprie prin care o colectie mare de fisiere header sunt parsate, iar informatiile sunt pastrate intr-un format specific care permite accesul rapid la informatiile continute in acele fisere header. In cazul in care nu este folosit acest "precompiled header", pentru fiecare fisier sursa ce trebuieste compilat toate fisierele header incluse sunt parcurse in mod independent, ceea ce consuma multe resurse (in special timp). Prin creearea acestei colectii de informatii (un fisier cu extensia .pch) se reduce timpul de compilare semnificativ. Toate fisiere .h des folosite (si care nu se modifica – de ex. Biblioteci de sistem, alte biblioteci de dimensiuni mari folosite) sunt include in acest fisier stdafx.h, iar includerea lui in fisierele sursa (cu #include "stdafx.h") duce la includerea (indirecta) a tuturor fisierelor header incluse in stdafx.h. Folosirea acestei tehnologii face obligatoriu ca prima linie utila (alta decat comentarii) din orice fisier sursa sa fie #include "stdafx.h", in caz contrar fiind generata o eroare. In mod independent se poate stabili folosirea sau nu a acestei tehnologii pentru fiecare fisier sursa din setarile proiectului.

2.2. Prezentarea mediului integrat



Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

In partea stanga (de obicei) se poate observa un control cu doua pagini (in figura). Aceste doua pagini ne confera acces facil si informatii despre proiect. “ClassView” contine clasele, structurile si functiile folosite in cadrul proiectului. Un click dreapta pe fiecare din intrarile de aici genereaza un meniu cu diferite optiuni, specifice fiecarei tip de intrare. Este oferita facilitatea modificarii modul in care sunt organizate intrarile, prin creearea unui sistem arborescent de foldere si aranjarea intrarilor cu drag-and-drop.

La FileView gasim lista fisierelor folosite in cadrul proiectului. Si aici avem meniuri dependente de entitate si posibilitatea organizarii lor.

In partea din dreapta se afla editorul. Ceea ce trebuie remarcat este facilitatea de “autocomplete”. Prin apasarea concomitenta a tastelor Ctrl si Space apare o lista cu optiuni pentru completarea numelor de variabile, functii, spatii de nume ceea ce duce la o viteza mai mare de tastare si reducerea erorilor de tastare a acestor nume.

In partea de jos avem informatii despre compilare, rezultatele cautarilor de/in fisiere, etc.

Adaugarea de noi fisiere la proiect se face astfel:

- selectare optiune File / New sau Ctrl + N
- alegerea tipului de fisier dorit din lista. Spre exemplu pentru fisiere header alegem “C/C++ Header file“ iar pentru fisiere sursa alegem “C++ Source file”
- introducerea numelui fisierului in editul “File name:“
- selectarea butonului “Ok“ din partea dreapta jos.

Adaugarea de fisiere existente la proiect se face astfel:

- Project / Add Project / Files
- selectarea fisier dorit din fereastra de dialog
- selectarea butonului “Ok“ din partea dreapta jos.

Compilarea/rularea se face folosind intrari din meniul “Build”. In cazul in care proiectul necesita compilarea si s-a ales rularea, compilarea se face automat.

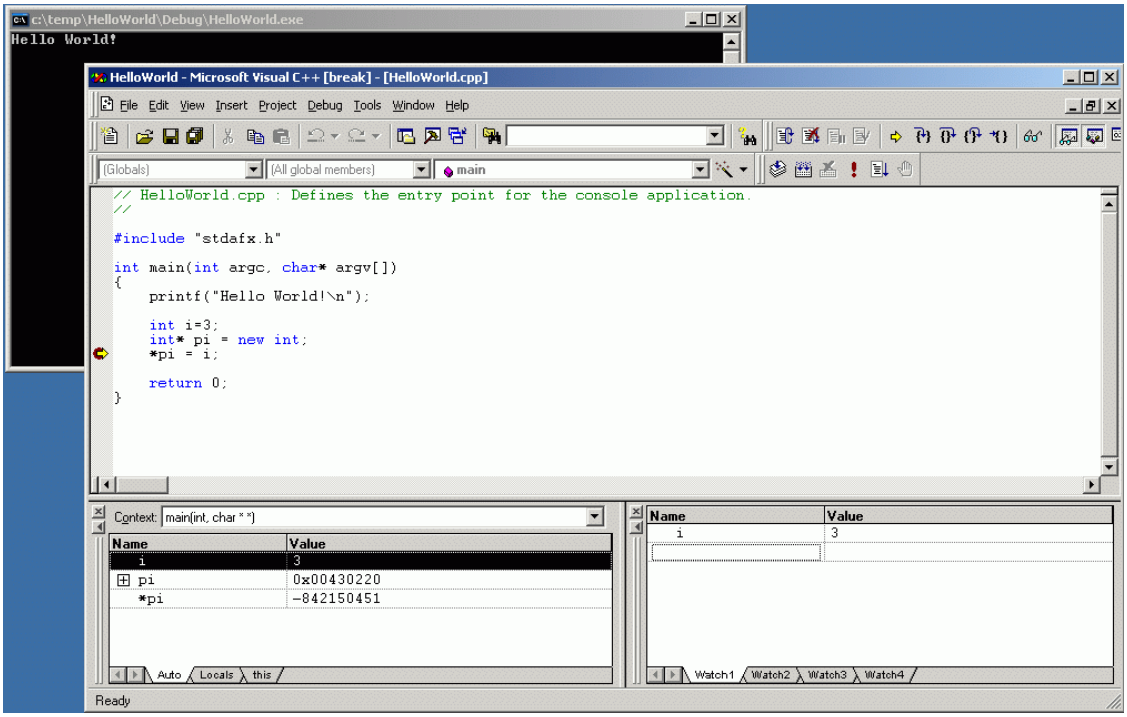
2.3. Debuger-ul

Debuger-ul prezent in acest mediu integrat este foarte puternic si usor de folosit.

Sa modificam programul adaugand urmatoarele linii inainte de return:

```
int i=3;
int* pi= new int;
*pi = i;
```

Pe una din linii se introduce un breakpoint (punct de oprire al programului) din meniu sau cu F9. Se ruleaza programul, executia oprindu-se cind se ajunge la acea linie:



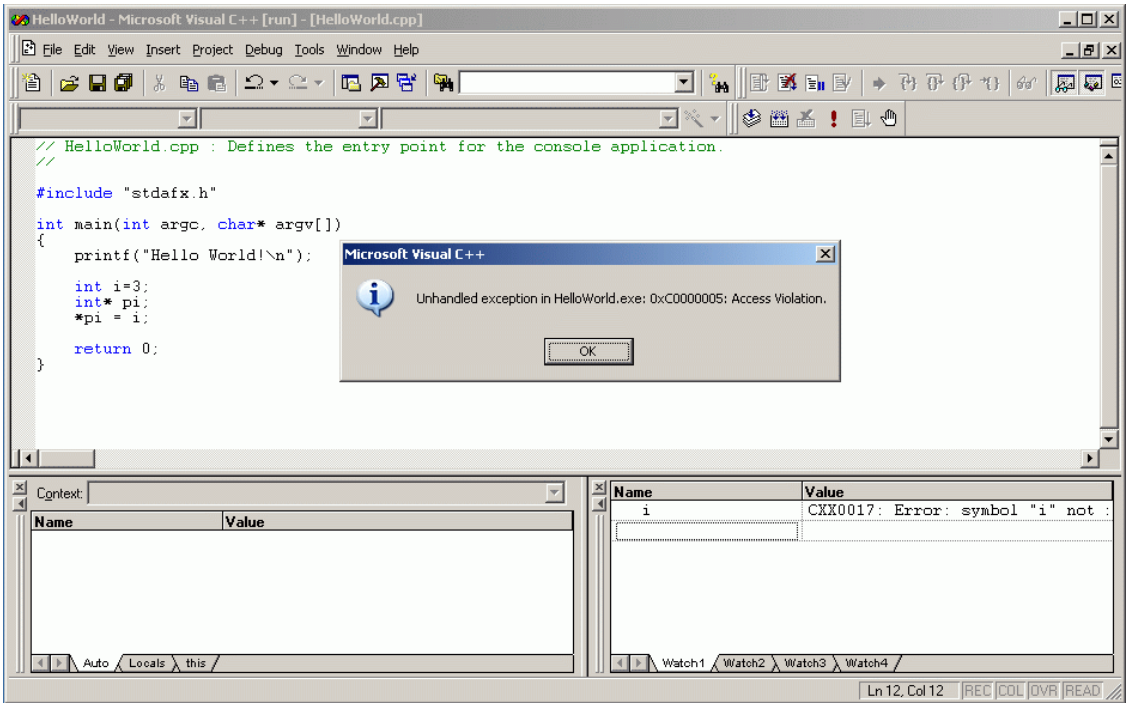
Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

Dupa cum se poate observa, avem mai multe zone pentru observarea variabilelor. In zona "auto" se vor afisa variabilele folosite in acel punct al programului (in mod automat). In zona "local" variabilele din acea functie. In zonele "watch" se pot adauga variabilele/expresiile dorite ce vor fi monitorizate pe tot parcusul rularii. In timpul rularii in mod debug apare un nou menu (Debug) cu intrarile uzuale pentru aceasta operatie (step in, step out, step over, run to cursor, etc). In meniul View/Debug windows pot fi adaugate/ascunse o serie de astfel de ferestre in care se afla informatii utile despre executia programului (memory, variables, call stack, registers, etc).

Una din alte facilitatile modului de debug este captarea diferitor exceptii ce pot apare la rularea programului. Ne vom opri la cele de tipul "access violation" (folosirea unei zone de memorie in mod impropriu). Sa incercam sa folosim pointerul pi fara a-l aloca in prealabil:

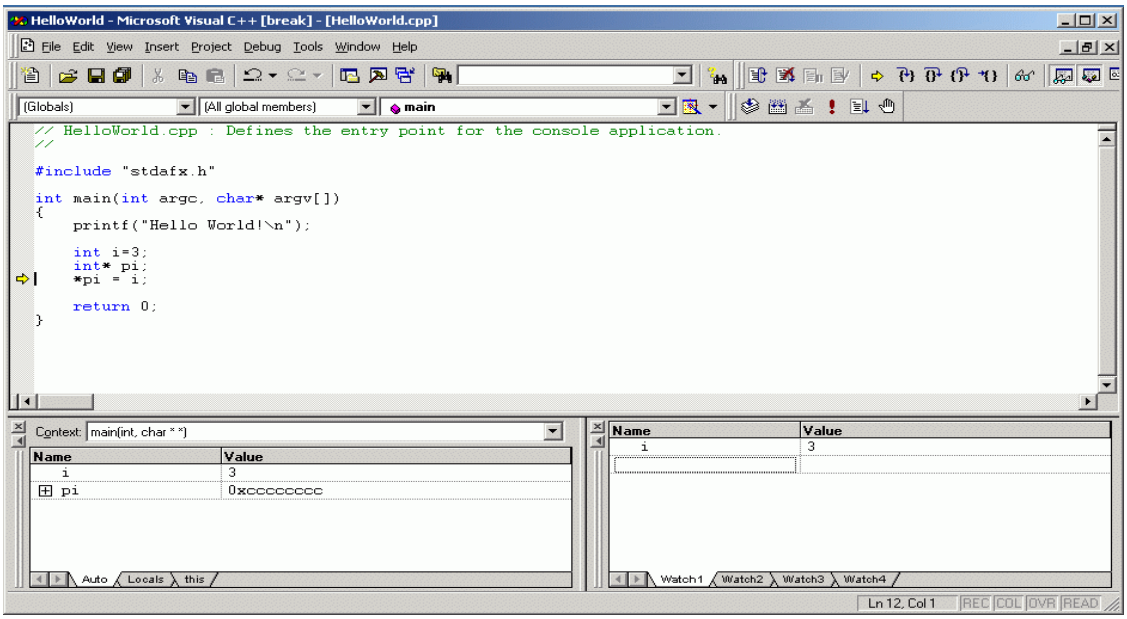
```
int i=3;
int* pi;
*pi = i;
```

La rularea va apare urmatoarea fereastra de avertizare:



Daca apasam “ok” vom fi dusi la linia de cod la care s-a produs exceptia (modul debug si stoparea executiei programului sunt facute automat).

Valorea “0xcccccccc” a lui pi denota faptul ca acel pointer nu a fost initializat (la rularea programului pi va avea o valoare aleatoare).



Exercitiu

Sa se scrie un program care citeste studentii dintr-o grupa si ii afiseaza. Programul va fi impartit in trei module:

Modulul Student

Interfata acestui modul va fi STUDENT.H :

```
#ifndef _STUDENT_
#define _STUDENT_

struct Student {
    char* nume;
    int   nota;
    };
```

```
void InitStudent (Student&);
void AfisStudent (Student);
void StergeStudent (Student&);

#endif
```

Implementarea (fisierul STUDENT.CPP) cuprinde:

■ InitStudent citeste numele studentului (pentru care va aloca spatiu cu malloc sau new) si nota.
■ AfisStudent afiseaza cimpurile structurii.
■ StergeStudent va elibera spatiul de memorie ocupat de nume (cu free sau delete).

Modulul Grupa

```
Interfata acestui modul va fi GRUPA.H :

  #ifndef _GRUPA_
  #define _GRUPA_

  #include "student.h"

  struct Grupa {
     Student* tab;
     int nr;
     int id;          // numarul grupei, de exemplu 1105
     };

  void InitGrupa (Grupa&);
  void AfisGrupa (Grupa);
  void StergeGrupa (Grupa&);

  #endif
```

Implementarea (fisierul GRUPA.CPP) cuprinde:

■ InitGrupa citeste numarul grupei si numarul de studenti, dupa care va aloca spatiu cu malloc pentru acestia. Fiecare student va fi apoi initializat cu InitStudent.
■ AfisGrupa afiseaza studentii.
■ StergeGrupa va elibera spatiul de memorie ocupat de cei *nr* studenti.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

Modulul Program principal

Se va gasi in fisierul MAIN.CPP:

```
  #include <stdio.h>
  #include "student.h"
  #include "grupa.h"

  void main()
  {
     Grupa g;
     InitGrupa(g);                      // citeste studentii
     AfisGrupa(g);                    // afiseaza grupa
     StergeGrupa(g);                  // elibereaza spatiul
}
```

Fisierele STUDENT.CPP, GRUPA.CPP, INDEX. CPP si MAIN.CPP se introduc intr-un proiect in Visual C++ 6.0. Programul demonstrativ este LAB1.EXE.

3. Complexitatea algoritmilor

3.1. Consideratii teoretice

La evaluarea (estimarea) algoritmilor secventiali se pune in evidenta necesarul de timp si de spatiu de memorare al lor.

Studierea complexitatii presupune analiza completa in cadrul algoritmului a urmatoarelor 3 apecte:

1. configuratia de date cea mai defavorabila (cazurile degenerate);
2. configuratia de date cea mai favorabila;
3. comportarea medie.

Comportarea medie presupune probabilitatea de aparitie a diferitelor configuratii de date la intrare. Aspectul 1 este cel mai studiat si este folosit, de obicei, pentru compararea algoritmulor.

Complexitatea unui algoritm secvential se exprima de regula in limbajul ordinului O.

Definitie
Fie f : N->N si g : N->N doua functii.
Spunem ca f apartine O(g) si notam f = O(g) daca si numai daca exista o constanta

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

reala c si un numar natural n_0 astfel incat pentru $n > n_0 \Rightarrow f(n) < c * g(n)$

Observatie:
f : N->N este o functie f(n) cu n dimensiunea datelor de intrare.
f(n) reprezinta timpul de lucru al algoritmului exprimat in "pasi".

Lema 1
Daca f este o functie polinomiala de grad k atunci $f = O(n^k)$.

Concluzie: $f = O(n^k)$, si ordinul O exprima viteza de variatie a functiei, functie de argument.

Proprietati:

1) Fie f, g : N->N.
Daca $f = O(g)$ $\quad k * f = O(g)$
$f = O(k * g)$, $k \in R$ constant.

2) Fie f, g, h : N->N.
si: $f = O(g)$
$g = O(h)$ atunci $f = O(h)$

3) Fie f1, f2, g1, g2 : N->N.
si: $f1 = O(g1) \Rightarrow f1 + f2 = O(g1 + g2)$
$f2 = O(g2) \Rightarrow f1 * f2 = O(g1 * g2)$

Aceasta proprietate permite ca, atunci cand avem doua bucle imbricate (de complexitati diferite), complexitatea totala sa se obtina inmultindu-se cele doua complexitati. Cele doua complexitati se aduna, daca buclele sunt succesive.

Teorema:
Oricare ar fi doua constante $c > 0$, $a > 1$, si $f : N \to N$, o functie monotona strict crescatoare, atunci:
$(f(n)) = O(a^{c\,f(n)})$

Intre clasa functiilor logaritmice, si cea a functiilor polinomiale exista relatia:
$O(n^c)$ inclusa in $O(a^n)$.

Au loc urmatoarele incluziuni:
O(1) in ***O(log n)*** in ***O(n)*** in ***O(nlog n)*** in ***O(n^2)*** in ... in ***O(n^k log n)*** in ***O(n^{k+1})*** in ***O(2^n)***

Pentru calculul complexitatii se va incerca incadrarea in clasa cea mai mica de complexitate din acest sir:

O(1) clasa algoritmilor constanti;
O(log n) clasa algoritmilor logaritmici;
O(n) clasa algoritmilor liniari;
O(nlog n) clasa algoritmilor polilogaritmici;
$O(n^2)$ clasa algoritmilor patratici;
$O(n^k \log n)$ clasa algoritmilor polilogaritmici;
$O(n^{k+1})$ clasa algoritmilor polinomiali;
$O(2^n)$ clasa algoritmilor exponentiali.

3.2. Algoritmul INSERTION SORT

Algoritmul INSERTION_SORT considera ca in pasul ***k***, elementele ***A[1÷k-1]*** sunt sortate, iar elementul ***k*** va fi inserat, astfel incat, dupa aceasta inserare, primele elemente ***A[1÷k]*** sa fie sortate.

Pentru a realiza inserarea elementului ***k*** in secventa ***A[1÷k-1]***, aceasta presupune:
- memorarea elementului intr-o varibila temporara;
- deplasarea tuturor elementelor din vectorul ***A[1÷k-1]*** care sunt mai mari decat ***A[k]***, cu o poziie la dreapta (aceasta presupune o parcurgere de la dreapta la stanga);
- plasarea lui ***A[k]*** in locul ultimului element deplasat.

Complexitate: ***O(n)***

PseudoCod:

```
insertion_sort(A,n)
{
     for k = 2 to n do
          temp = A[k]
          i=k-1
          while(i >= 1) and (A[i] > temp) do
                A[i+1] = A[i]
                i = i-1
          A[i+1] = temp
}
```

Cazul cel mai dafavorabil: situatia in care deplasarea (la dreapta cu o pozitie in vederea inserarii) se face pana la inceputul vectorului, adica sirul este ordonat descrescator.

Exprimarea timpului de lucru:
T(n) = 3 (n - 1) + 3 (1 + 2 + 3+ ... + n - 1) = 3(n-1) + 3n * (n - 1)/2
Rezulta complexitatea: $\boldsymbol{T(n) = O(n^2)}$ functie polinomiala de gradul II.

Observatie: Cand avem mai multe bucle imbricate, termenii buclei celei mai interioare dau gradul polinomului egal cu gradul algoritmului.

Bucla cea mai interioara ne da complexitatea algoritmului.

3.3. Cautarea binara (Binary Search)

Fie ***A***, de ordin ***n***, un vector ordonat crescator. Se cere sa se determine daca o valoare ***b*** se afla printre elementele vectorului. Limita inferioara se numeste ***low***, limita superioara se numeste ***high***, iar mijlocul virtual al vectorului, ***mid*** (de la middle).

PseudoCod:

```
Binary_search(A,n,b)
{
     low = 1
     high = n
     while low <= high do
          mid = [(low + high)/2]         // partea intreaga
          if A[mid] = b then return mid
          else if A[mid] > b then
                  high = mid-1  // restrang cautarea
                                // la stanga
               else low = mid+1 // restrang cautarea
                                // la dreapta
     return(0)
}
```

Calculul complexitatii algoritmului consta in determinarea numarului de ori pentru care se executa bucla *while*. Se observa ca, la fiecare trecere, dimensiunea zonei cautate se injumatateste.

Cazul cel mai defavorabil este ca elementul cautat sa nu se gaseasca in vector.

Pentru simplitate se considera $n = 2^k$, unde k este numarul de injumatatiri.

Rezulta $k = \log_2 n$ si facand o majorare, $T(n) <= \log_2 n + 1$, a.i.

$2 * 2^k <= n < 2^{k+1}$.

Rezulta complexitatea acestui algoritm: este $\boldsymbol{O(\log_2 n)}$. Dar, baza logaritmului se poate ignora, deoarece: $\log_a x = \log_a b * \log_b x$ si $\log_a b$ este o constanta, deci ramane $\boldsymbol{O(\log n)}$, adica o functie logaritmica.

Laborator de Structuri de Date si Algoritmi – Lucrarea nr. 1

TEMA

Sa se construiasca un modul (fisierle .H si .C (.CPP)) care sa contina tipurile de date si functiile care implementeaza algoritmii analizati anterior si calculeaza numarul de operatii (atribuiri si comparatii) executate pentru diferite instante de intrare.

Sa se construiasca un program C (C++) care cu ajutorul unui meniu simplu sa permita urmatoarele:

- introducerea datelor
- sortarea prin metoda insertiei si afisarea numarului de operatii
- cautarea prin metoda cautarii binare a unei valori introduse de la tastatura si afisarea numarului de operatii